GUIDE 08

Develop React Frontends for Listings and Reviews

UniMart Monolithic Full-Stack Lab Series

B.Sc. (Hons) IT - Level 2 | Hands-on Laboratory Practical

Department of Industrial Management
Faculty of Science, University of Kelaniya

Learning outcomes

- Build the listing and review user interfaces against the agreed API contract.
- Use RTK Query for server state and Redux slices for client/auth state.
- Create accessible Material UI forms and responsive Tailwind layouts.
- Handle loading, empty, validation, authorization and error states.

Prerequisites

- Guide 02 frontend setup completed.
- Guide 07 API running locally and Postman tests passing.

Suggested student time: 3-4 hours across two lab sessions

1. Route and page plan

Route	Page	Main behavior
/	ListingsPage	Search/filter/paginate available listings.
/listings/new	ListingFormPage	Authenticated seller creates a listing.
/listings/:id	ListingDetailsPage	Show listing, seller and reviews.
/listings/:id/edit	ListingFormPage	Owner edits listing.
/orders/:orderId/review	ReviewFormPage	Buyer reviews a completed order.
/my/listings	MyListingsPage	Owner sees edit/archive actions.
/login	LoginPage	Authenticate and initialize access token state.

2. TypeScript domain types

```[TypeScript]
export type ListingStatus = 'AVAILABLE' | 'RESERVED' | 'SOLD' | 'ARCHIVED';

export interface Listing {
  id: number;
  sellerId: number;
  sellerName: string;
  categoryId: number;
  categoryName: string;
  title: string;
  description: string;
  price: number;
  status: ListingStatus;
  createdAt: string;
  updatedAt: string;
}

export interface ListingInput {
  title: string;
  description: string;
  price: number;
  categoryId: number;
}

export interface Page<T> {
  content: T[];
  number: number;
  size: number;
  totalElements: number;
  totalPages: number;
}
```

```[TypeScript]
export interface Review {
  id: number;
  orderId: number;
  reviewerName: string;
  revieweeId: number;
  rating: number;
  comment: string | null;
  createdAt: string;
  updatedAt: string;
}
```

3. Create the RTK Query base API

```[TypeScript]
// src/services/baseApi.ts
import { createApi, fetchBaseQuery } from '@reduxjs/toolkit/query/react';
import type { RootState } from '../app/store';

export const baseApi = createApi({
  reducerPath: 'api',
  baseQuery: fetchBaseQuery({
    baseUrl: import.meta.env.VITE_API_BASE_URL,
    prepareHeaders: (headers, { getState }) => {
      const token = (getState() as RootState).auth.accessToken;
      if (token) headers.set('authorization', `Bearer ${token}`);
      headers.set('accept', 'application/json');
      return headers;
    },
    credentials: 'include',
  }),
  tagTypes: ['Listing', 'Review'],
  endpoints: () => ({}),
});
```

4. Listing endpoints

```[TypeScript]
export const listingsApi = baseApi.injectEndpoints({
  endpoints: (build) => ({
    getListings: build.query<Page<Listing>, ListingQuery>({
      query: (params) => ({ url: 'listings', params }),
      providesTags: (result) => result
        ? [
            ...result.content.map(({ id }) => ({ type: 'Listing' as const, id })),
            { type: 'Listing', id: 'LIST' },
          ]
        : [{ type: 'Listing', id: 'LIST' }],
    }),
    getListing: build.query<Listing, number>({
      query: (id) => `listings/${id}`,
      providesTags: (_r, _e, id) => [{ type: 'Listing', id }],
    }),
    createListing: build.mutation<Listing, ListingInput>({
      query: (body) => ({ url: 'listings', method: 'POST', body }),
      invalidatesTags: [{ type: 'Listing', id: 'LIST' }],
    }),
    updateListing: build.mutation<Listing, { id: number; body: ListingInput }>({
      query: ({ id, body }) => ({ url: `listings/${id}`, method: 'PUT', body }),
      invalidatesTags: (_r, _e, { id }) => [
        { type: 'Listing', id },
        { type: 'Listing', id: 'LIST' },
      ],
    }),
    archiveListing: build.mutation<void, number>({
      query: (id) => ({ url: `listings/${id}`, method: 'DELETE' }),
      invalidatesTags: (_r, _e, id) => [
        { type: 'Listing', id },
        { type: 'Listing', id: 'LIST' },
      ],
    }),
  }),
});
```

5. Responsive listing card

```tsx
export function ListingCard({ listing }: { listing: Listing }) {
  return (
    <Card className="h-full">
      <CardContent className="flex h-full flex-col gap-3">
        <div className="flex items-start justify-between gap-3">
          <Typography variant="h6" component="h2">
            {listing.title}
          </Typography>
          <Chip size="small" label={listing.status} />
        </div>
        <Typography color="text.secondary">
          {listing.categoryName}
        </Typography>
        <Typography className="line-clamp-3" sx={{ flexGrow: 1 }}>
          {listing.description}
        </Typography>
        <Typography variant="h6">
          LKR {listing.price.toLocaleString('en-LK', { minimumFractionDigits: 2 })}
        </Typography>
        <Button component={Link} to={`/listings/${listing.id}`} variant="contained">
          View listing
        </Button>
      </CardContent>
    </Card>
  );
}
```

6. Listing form with React Hook Form and Zod

```tsx
const listingSchema = z.object({
  title: z.string().trim().min(1).max(160),
  description: z.string().trim().min(1).max(5000),
  price: z.coerce.number().min(0),
  categoryId: z.coerce.number().int().positive(),
});

type ListingFormValues = z.infer<typeof listingSchema>;

export function ListingForm({ initial, onSubmit }: Props) {
  const { register, handleSubmit, formState: { errors, isSubmitting } } =
    useForm<ListingFormValues>({
      resolver: zodResolver(listingSchema),
      defaultValues: initial,
    });

  return (
    <Box component="form" onSubmit={handleSubmit(onSubmit)}
         className="mx-auto grid max-w-2xl gap-4">
      <TextField label="Title" error={!!errors.title}
        helperText={errors.title?.message} {...register('title')} />
      <TextField label="Description" multiline minRows={5}
        error={!!errors.description}
        helperText={errors.description?.message}
        {...register('description')} />
      <TextField label="Price (LKR)" type="number"
        inputProps={{ min: 0, step: '0.01' }}
        error={!!errors.price}
        helperText={errors.price?.message}
        {...register('price')} />
      <Button type="submit" variant="contained" disabled={isSubmitting}>
        Save listing
      </Button>
    </Box>
  );
}
```

7. Reviews endpoints and form

```[TypeScript]
export const reviewsApi = baseApi.injectEndpoints({
  endpoints: (build) => ({
    getListingReviews: build.query<Page<Review>, { listingId: number; page?: number }>({
      query: ({ listingId, page = 0 }) => ({
        url: `listings/${listingId}/reviews`, params: { page, size: 10 },
      }),
      providesTags: (_r, _e, { listingId }) =>
        [{ type: 'Review', id: `LISTING-${listingId}` }],
    }),
    createReview: build.mutation<Review, ReviewCreateInput>({
      query: (body) => ({ url: 'reviews', method: 'POST', body }),
      invalidatesTags: (_r, _e, arg) =>
        [{ type: 'Review', id: `LISTING-${arg.listingId}` }],
    }),
    updateReview: build.mutation<Review, { id: number; body: ReviewUpdateInput }>({
      query: ({ id, body }) => ({ url: `reviews/${id}`, method: 'PUT', body }),
      invalidatesTags: ['Review'],
    }),
    deleteReview: build.mutation<void, number>({
      query: (id) => ({ url: `reviews/${id}`, method: 'DELETE' }),
      invalidatesTags: ['Review'],
    }),
  }),
});
```

```[TSX]
<Controller
  control={control}
  name="rating"
  render={({ field }) => (
    <Rating
      value={field.value}
      onChange={(_event, value) => field.onChange(value ?? 0)}
      aria-label="Review rating"
    />
  )}
/>
<TextField
  label="Comment"
  multiline
  minRows={4}
  inputProps={{ maxLength: 1000 }}
  {...register('comment')}
/>
<Button type="submit" variant="contained">Submit review</Button>
```

8. UI states that must be implemented

State	Required UI
Loading	Skeleton cards or progress indicator with stable layout.
Empty	Explain that no listings/reviews match; provide clear next action.
Validation	Show field-specific messages before submission and server validation after submission.
401	Attempt refresh/login flow; do not display raw server stack traces.
403	Explain ownership/permission requirement and hide actions the user cannot take.
404	Friendly not-found page with link back to listings.
409	Show business conflict, such as duplicate review or sold listing.
Network error	Retry control and offline-friendly message.
Success	Toast/snackbar and navigation or cache refresh.

9. Accessibility and security checks

- Every input has a visible label; icon-only buttons have aria-label.
- Keyboard focus order is logical and dialogs return focus after closing.
- Do not use dangerouslySetInnerHTML for listing descriptions or reviews.
- Do not display or persist password/token values in forms, logs or Redux DevTools.
- Hide unauthorized controls for usability, but rely on backend authorization for security.
- Use semantic headings and sufficient contrast; test at mobile and desktop widths.

10. Frontend tests and acceptance evidence

- Component test: ListingCard renders title, price and link.
- Form test: invalid title and rating are rejected.
- Integration test: successful create mutation invalidates the listing list cache.
- Manual test: owner sees edit/archive; non-owner does not.
- Browser Network evidence: correct URL, method, bearer header and JSON response.
- Production build succeeds with npm run build.

Completion checklist

☐ Listings page supports loading, search/filter, empty and error states.

☐ Create/edit listing form validates the same limits as backend DTOs.

☐ Listing details page displays reviews.

☐ Completed-order review form supports create/update/delete policy.

☐ RTK Query tags refresh changed data without full page reload.

☐ MUI and Tailwind are used consistently and accessibly.